

IT362: Principles of Data Science

How AI Is Reshaping Developer Language Choices Over Time

Prepared By

Section	Student Name	ID
80657	Afnan Alkharji	443200897
	Deema Alfarhoud	444200955
	Hussah Alotaibi	444201039
	Falwa alkhalifah	444200745
	Sana Alotaibi	444200426

Supervised by:

Dr. Abeer Aldayel

Contents

1. Phase 1: Data Collection Research and Assessment	3
i. Introduction	3
ii. Dataset	3
iii. Objectives	5
iv. Method.....	6
v. Challenges faced in data collection and recommendations	6
2. Phase 2: Data Processing, Cleaning, and Exploratory Data Analysis (EDA)	7
i. Data Processing and cleaning.....	7
ii. Exploratory Data Analysis (EDA).....	8
3. Phase 3: Modelling and Communication	17
i. Data Preparation	17
ii. Clustering Approach.....	17
iii. Regression Results.....	18
iv. Hypothesis Testing and Findings.....	18
4. Conclusion and Recommendations	19

1. Phase 1: Data Collection Research and Assessment

This phase involved gathering the raw GitHub repository data, resolving API limitations, and merging all yearly files into a unified dataset. The goal was to ensure the data was complete, consistent, and ready for preprocessing.

i. Introduction

The rise of large language models, such as ChatGPT, has significantly impacted the field of artificial intelligence. This project investigates whether this shift has also affected the programming languages used in open-source AI development.

By collecting and analyzing metadata from popular AI-related GitHub repositories before and after the release of ChatGPT, we aim to examine trends in language usage over time. The goal is to determine if the growth of generative AI has influenced developers' preferences for programming languages within the AI community.

ii. Dataset

Data Source

The data source for this study is the GitHub REST API. Which provides a programmatic interface to access repositories information.

Data Collection

```
# ==== CONFIG ====
GITHUB_TOKEN = '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'
HEADERS = {
    "Authorization": f"token {GITHUB_TOKEN}",
    "Accept": "application/vnd.github+json",
    "X-GitHub-API-Version": "2022-11-28",
    "User-Agent": "Data-Science-Project-KSUM"
}
```

The dataset will be collected over a period spanning six years, from 2020 to 2025. For each year, we will identify and collect the top 1000 most-starred AI related repositories distributed over quarters (250 per quarter) to guarantee we capture the trends over the whole year.

```

QUERY = 'AI OR artificial-intelligence OR machine-learning OR generative-ai'
REPOS_PER_QUARTER = 250 # target per quarter
QUARTERS = [
    ('2024-01-01', '2024-03-31'),
    ('2024-04-01', '2024-06-30'),
    ('2024-07-01', '2024-09-30'),
    ('2024-10-01', '2024-12-31')
]

```

This means we collected approximately 1000 observations for each year, except for 2025 where we collected repositories up to 23rd of September and it resulted in 754 repositories. This will result in a total of approximately 5750 observations for our analysis.

Data Features

The following features will be collected for each repository:

Feature	Data type	Description
name	Qualitative (Nominal)	Name of the repository
owner	Qualitative (Nominal)	User or organization that owns the repository.
url	Qualitative (Nominal)	URL of the repository
description	Qualitative (Nominal)	A brief description of the project
stars	Quantitative (Ratio)	Number of times the repository has been starred
forks	Quantitative (Ratio)	Number of times the repository has been forked
created_at	Qualitative (Ordinal)	Date of the repository's creation
pushed_at	Qualitative (Ordinal)	Date of the last update or push to the repository
primary_language	Qualitative (Nominal)	Main programming language of the repository
all_languages_bytes	Qualitative (Nominal)	A dictionary showing the breakdown of all programming languages used in the repository and the number of bytes for each
topics	Qualitative (Nominal)	A list of topics or tags associated with the repository
contributors_count:	Quantitative (Ratio)	Number of unique contributors to the repository

Dataset Bias Evaluation

- **Representation Bias:** By focusing on the "most-starred" repositories, our dataset represents projects that have gained most attention, yet their fame could be due to effective marketing, or a high-profile creator and not necessarily trendy.
- **Measurement Bias:** By dividing trends by quarter, we ensure each period is represented. However, a "smaller" trend in an active quarter could be more impactful than a "main" trend in a less active quarter.
- **Historical Biases:** By analyzing existing repositories, we could be missing projects that were more popular during their time but have since been removed.

iii. Objectives

Our project aims to uncover “**how the emergence of large language models (LLMs), such as ChatGPT, has influenced programming language preferences in open-source AI development.**” Specifically, the objectives are to:

1. **Analyze Temporal Trends**
Examine changes in the frequency and proportion of programming languages used in popular AI-related GitHub repositories before and after the release of ChatGPT.
2. **Identify Shifts in Developer Preferences**
Detect any significant increases or decreases in the use of specific languages (e.g., Python, Java, Julia, R, C++) that may be associated with the rise of generative AI.
3. **Compare Pre- and Post-ChatGPT Periods**
Statistically compare programming-language usage in the two time windows to evaluate whether the advent of LLMs correlates with measurable changes in developer behavior.
4. **Characterize the AI Development Ecosystem**
Provide a descriptive overview of the most commonly used languages, their growth or decline rates, and how these patterns reflect evolving practices in AI research and engineering.
5. **Generate Insights for Future AI Tooling**
Offer evidence-based insights that can guide educators, practitioners, and technology strategists in understanding which programming languages are gaining traction in the era of generative AI.

iv. Method

The dataset collected from GitHub will be used to analyze trends in programming languages among AI-related projects over multiple years (2020–2025). Each repository includes information such as primary programming language, all languages used, number of stars, forks, topics, and contributors.

Using this dataset, the following steps will be taken:

1. **Data Cleaning & Preprocessing:** Ensure all entries have consistent formats, handle missing values, and normalize language names if needed.
2. **Descriptive Analysis:** Calculate KPIs such as the most common programming languages per year, average stars per language, and number of AI-related repositories created annually.
3. **Trend Analysis:** Compare language popularity across years to identify shifts in which languages are used in AI projects over time.
4. **Visualization:** Create charts and graphs to illustrate language trends, repository activity, and other key metrics.
5. **Insights:** Draw conclusions about how AI development trends have influenced programming language adoption year by year.

v. Challenges faced in data collection and recommendations

Challenges:

1. **API Rate Limits:** GitHub restricts the number of requests per hour. Without a valid token, the script could fail if too many requests are made.
2. **Time-Consuming Collection:** Fetching detailed data for many repositories takes several minutes per quarter.

Recommendations:

- Always use a valid personal access token to avoid hitting rate limits.
- For future projects, consider storing raw data snapshots at regular intervals to avoid repeating long collection processes.

2. Phase 2: Data Processing, Cleaning, and Exploratory Data Analysis (EDA)

This phase focused on cleaning and preparing the dataset by handling missing values, transforming features, and performing initial exploratory analysis to ensure the data was suitable for modeling.

i. Data Processing and cleaning

- **Checking duplicate data and missing values**

As part of the initial cleaning process, the dataset was examined for duplicate entries and missing values. We confirmed that the dataset contained no duplicate rows, but some missing values were identified particularly within text-based fields such as description and topics. However, only rows with missing primary programming language were removed, as this variable is essential for language-trend analysis.

- **Data transformation**

- 1- Conversion of Stored Numeric Values from String to Numeric Types

Some numerical features retrieved from the GitHub API were initially stored as strings. These fields were converted into proper numeric data types.

- 2- Text Feature Preparation

To ensure compatibility with text-processing techniques, missing values in fields such as description and topics were replaced with empty strings (“”).

- **Checking dataset size**

To ensure the dataset was large enough to support reliable statistical analysis, a sample size adequacy check was performed. Using the standard sample size formula with a 95% confidence level and a 5% margin of error.

The dataset comfortably exceeded the minimum required size, allowing the team to proceed confidently with Exploratory Data Analysis (EDA).

ii. Exploratory Data Analysis (EDA)

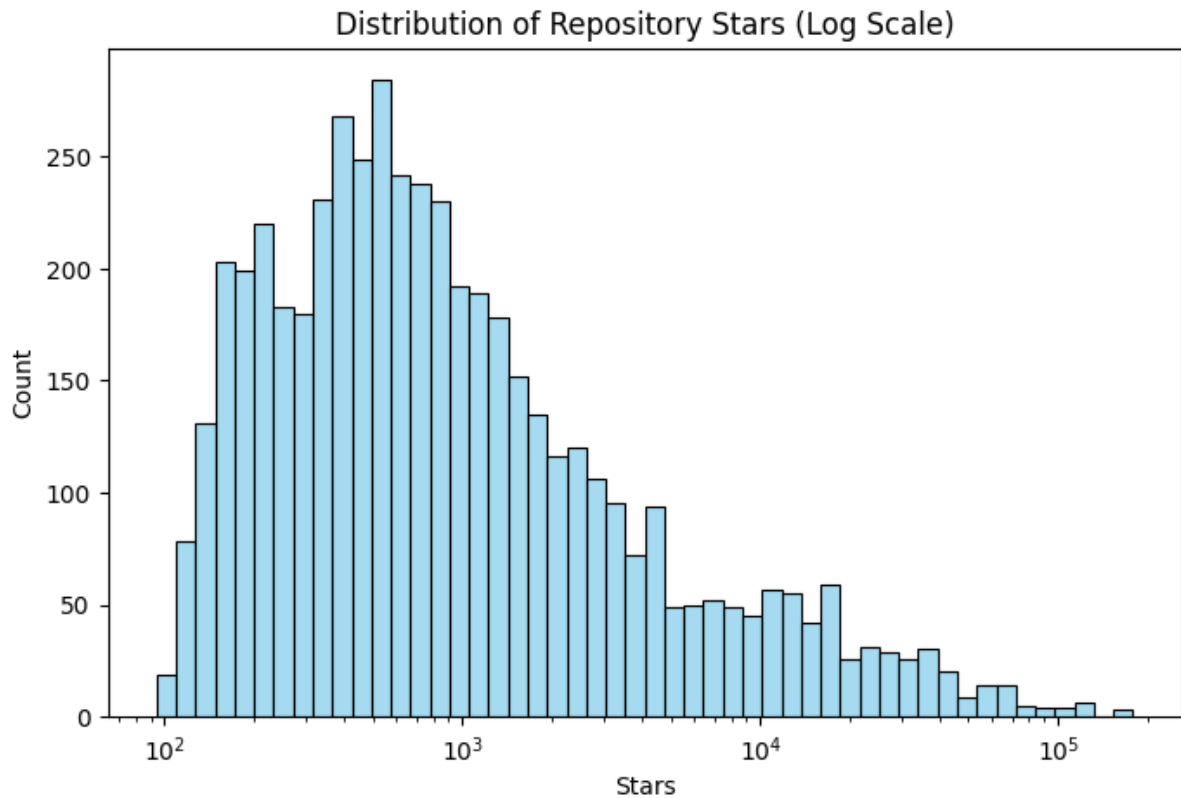


Figure 1: The first figure shows the logarithmic distribution of GitHub repository stars. The x-axis represents the number of stars on a log scale, while the y-axis shows the count of repositories within each range.

Analysis:

Most repositories have between 100 and 1,000 stars, while only a few reach very high star counts above 10,000. This right-skewed distribution indicates that a small number of projects gain significant popularity, whereas the majority receive moderate attention.

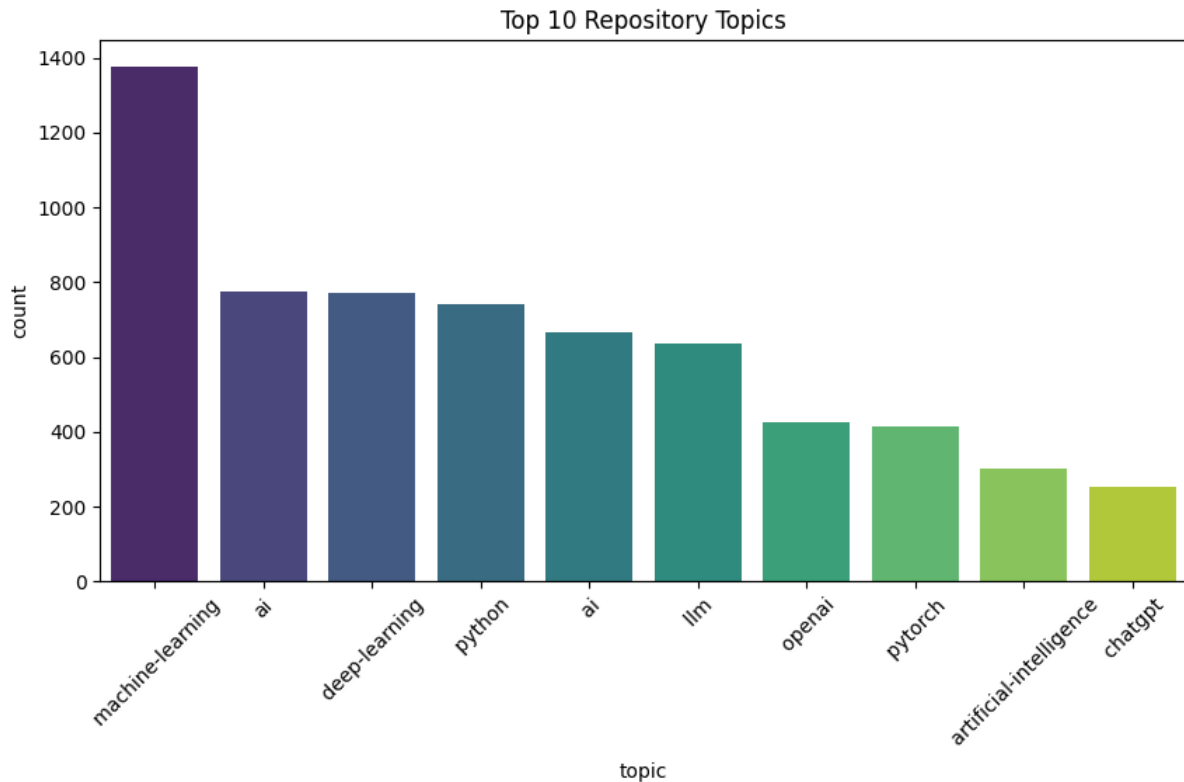


Figure 2: This bar chart shows the ten most common topics among repositories. “Machine learning” dominates, followed by “AI,” “deep learning,” and “Python.”

Analysis:

The results highlight that most popular repositories are centered around machine learning and AI technologies. This indicates strong community interest in artificial intelligence and related tools such as Python and PyTorch.

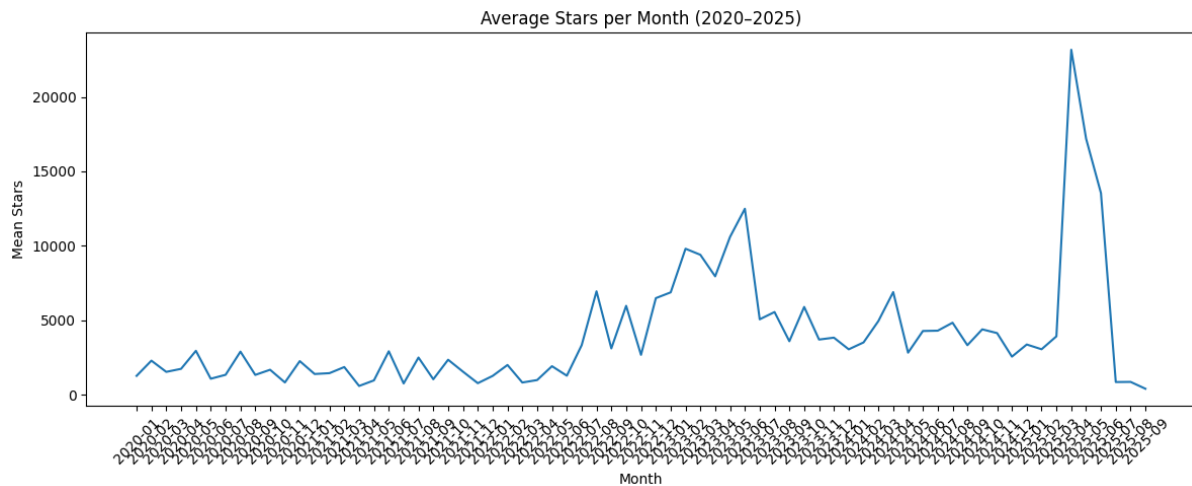


Figure 3: The line chart presents the average number of stars received by repositories each month from 2020 to 2025.

Analysis:

There is a noticeable surge in stars starting in late 2022, peaking sharply in 2023. This spike likely corresponds to the public rise of AI tools such as ChatGPT, reflecting increased attention and engagement with AI-related projects on GitHub.

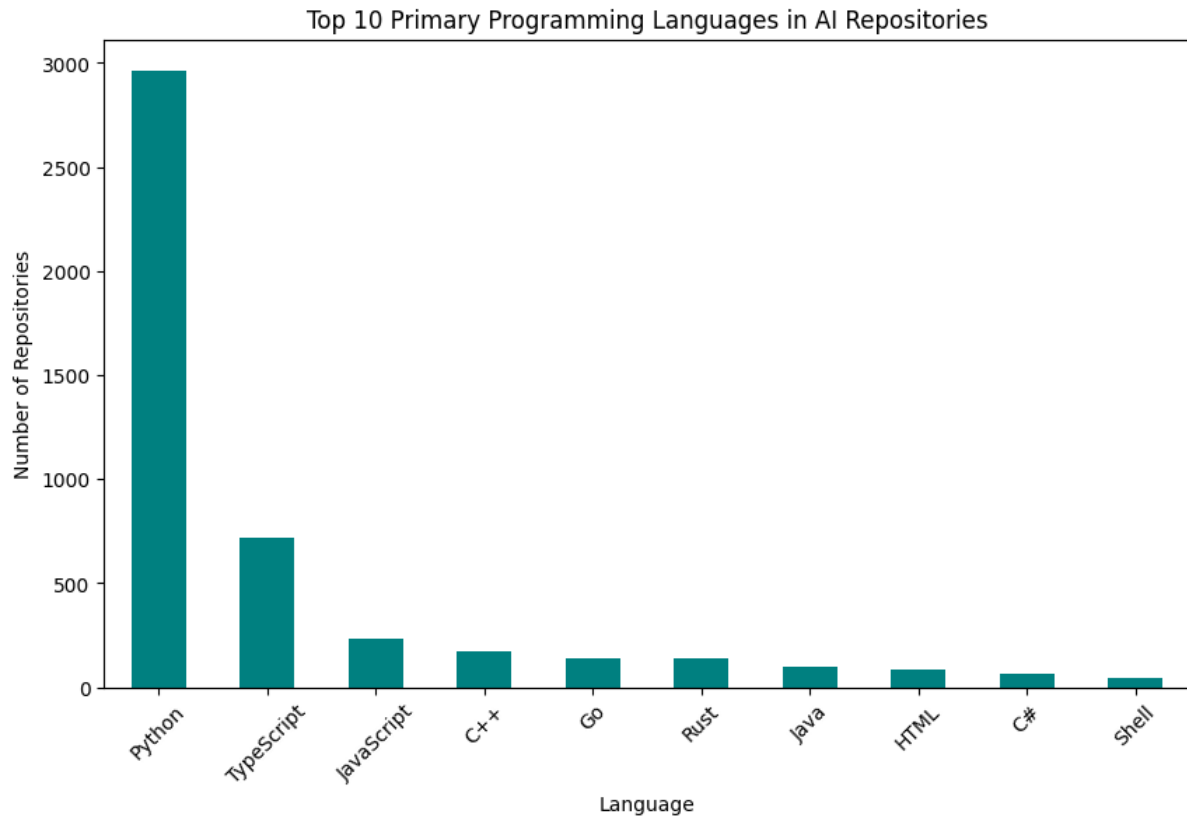


Figure 4: This bar chart displays the most common primary programming languages used in AI-related repositories on GitHub.

Analysis:

Python overwhelmingly dominates as the main language, far exceeding all others with nearly 3,000 repositories. The next most frequent languages are TypeScript and JavaScript, followed by C++ and Go. This confirms Python's central role in AI development due to its strong libraries and community support, while other languages play smaller, complementary roles in specialized or web-based applications.

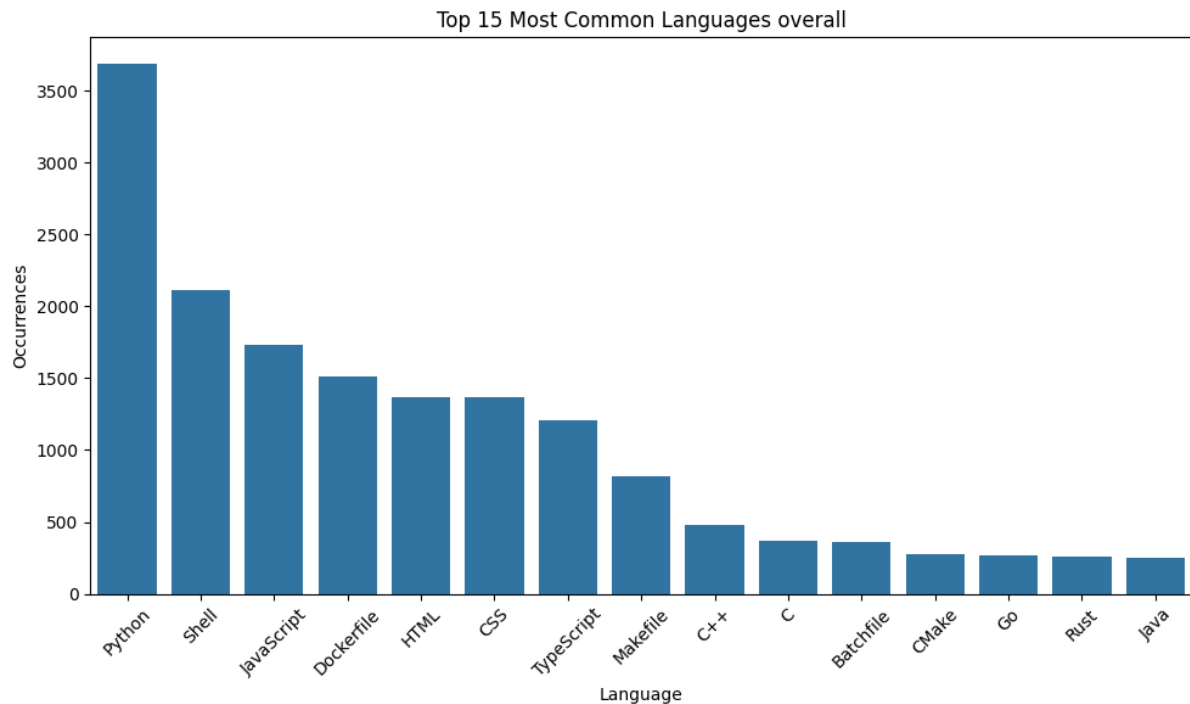


Figure 5: The bar chart illustrates the 15 most frequently used programming languages across all repositories.

Analysis:

Python leads by a wide margin, followed by Shell, JavaScript, and Dockerfile. The results confirm Python's dominance beyond AI projects, showing it as the most popular overall development language. Shell and Dockerfile also appear frequently, reflecting the importance of automation and deployment tools in modern repositories.

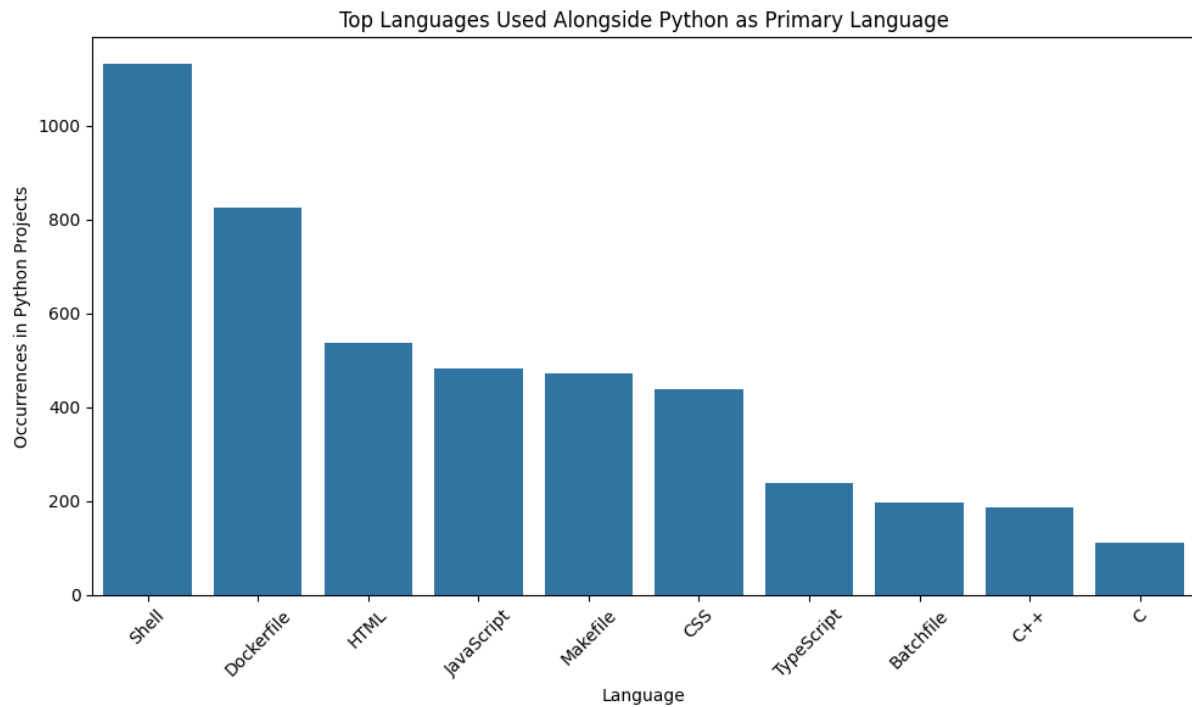


Figure 6: This figure shows which additional languages are most commonly found in repositories where Python is the main language.

Analysis:

Shell and Dockerfile are the most frequent companions to Python, followed by HTML, JavaScript, and Makefile. This combination suggests that Python projects often integrate scripting, containerization, and web development elements, highlighting Python's versatility and interoperability within diverse development environments.

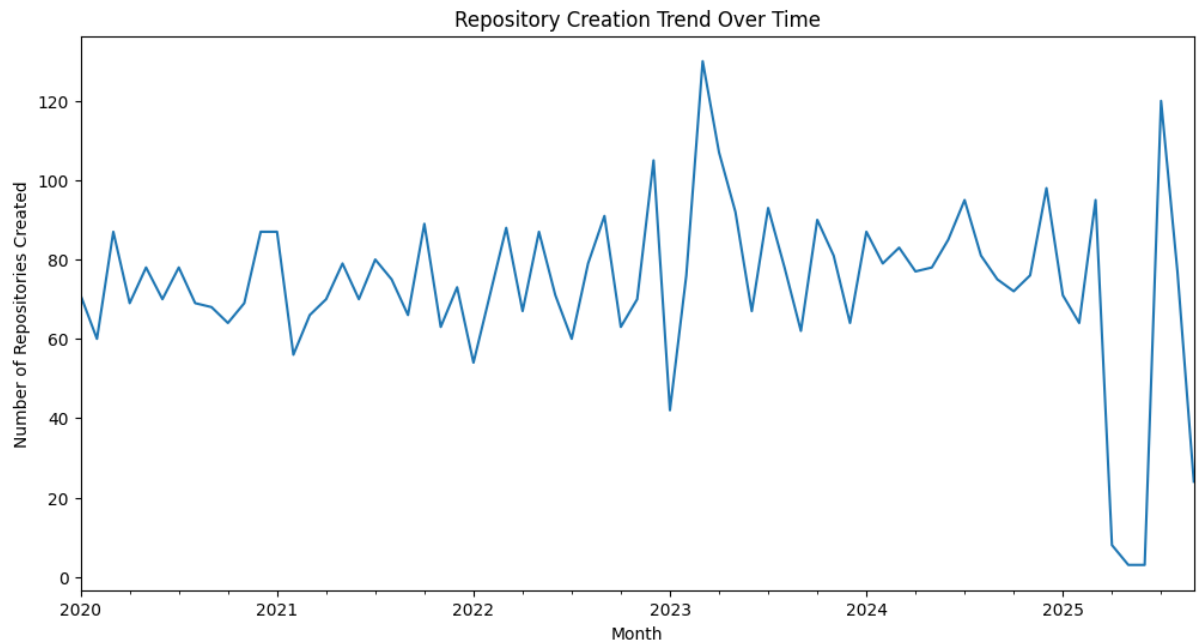


Figure 7: This line chart illustrates the number of repositories created each month from 2020 to 2025.

Analysis:

The trend remains relatively stable over time, with minor fluctuations and occasional spikes. A noticeable increase appears around late 2022 to early 2023, suggesting heightened activity in repository creation—likely influenced by the growing popularity of AI technologies during that period. The later drop may indicate incomplete data for the most recent months rather than an actual decline in development.

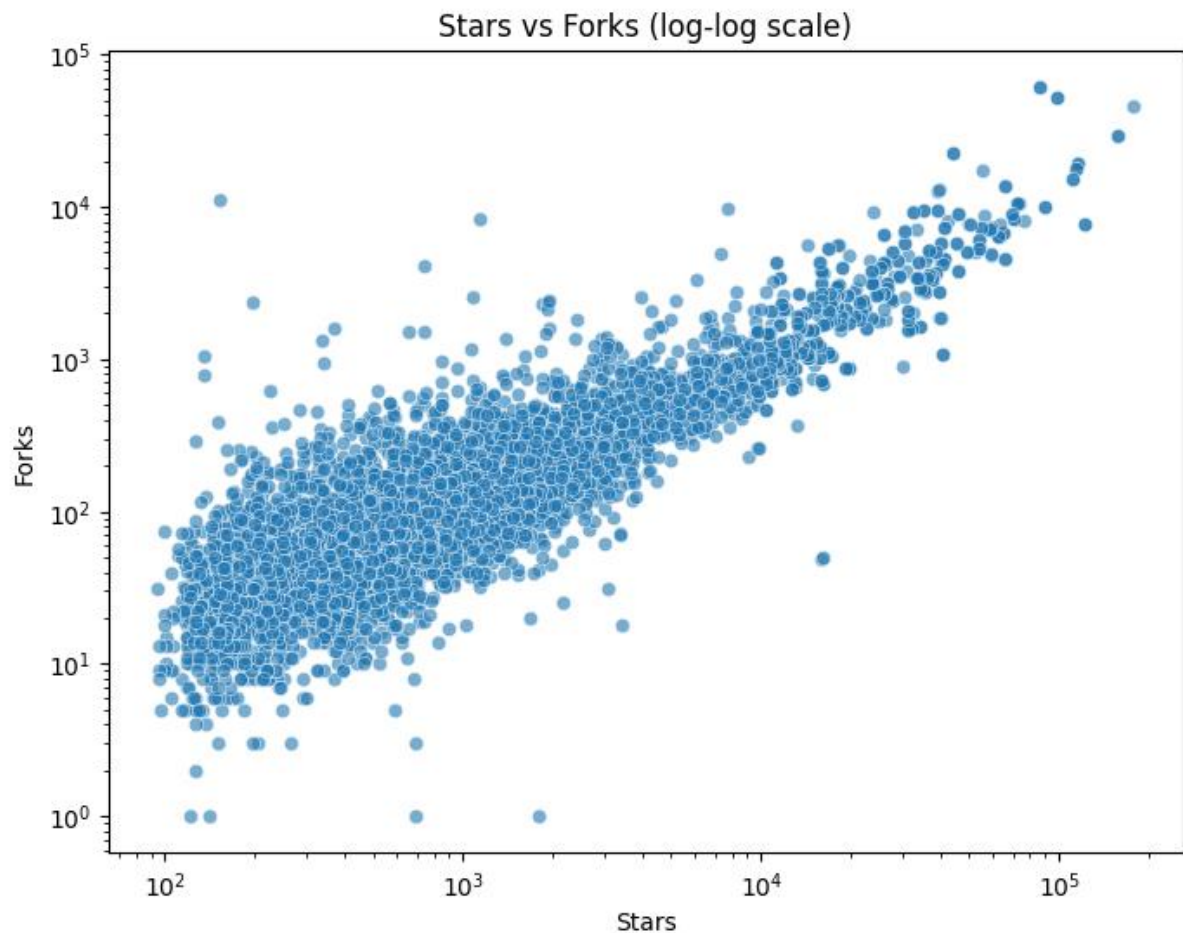


Figure 8: This scatter plot shows the relationship between the number of stars and forks for repositories, using a logarithmic scale on both axes.

Analysis:

There is a clear positive correlation between stars and forks — repositories with more stars tend to have more forks. This suggests that popular projects not only attract attention but also encourage community contributions and reuse, reflecting active engagement and collaboration within the developer community.

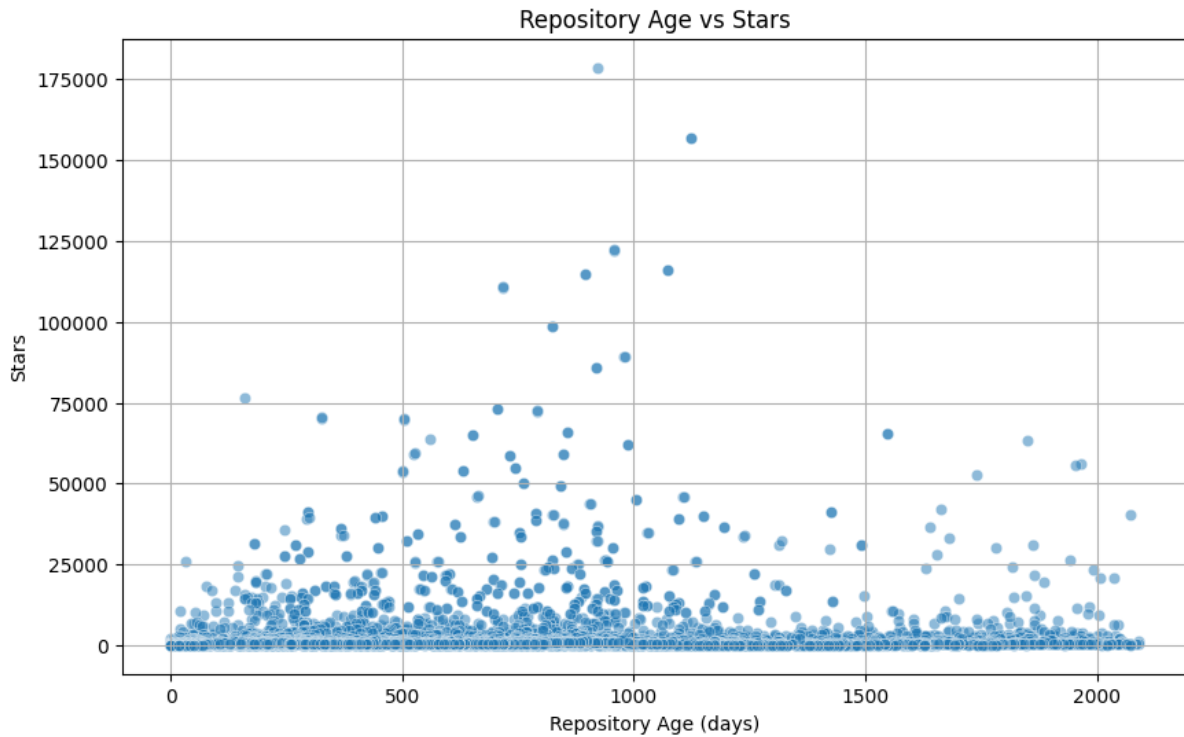


Figure 9: This scatter plot displays the relationship between repository age (in days) and the number of stars each project has received.

Analysis:

The chart shows no strong correlation between age and popularity. While older repositories have had more time to accumulate stars, several newer ones also gained significant attention quickly. This suggests that popularity is influenced more by project relevance and quality than by age, especially in rapidly evolving fields like AI.

3. Phase 3: Modelling and Communication

This phase utilized multiple machine learning and statistical methods to explore the characteristics of the GitHub repository data and quantify the impact of the ChatGPT release. This work included Clustering, Regression, and Parametric Hypothesis Testing.

i. Data Preparation

The data preparation steps were tailored for each specific modelling task:

- **Temporal Segmentation:** A binary column, period, was derived from the created_at timestamp using November 30, 2022 (ChatGPT's release date) as the cutoff . This allowed for pre- and post-ChatGPT comparisons.
- **Feature Selection:** For the core analysis, the features representing repository popularity and metadata were retained: stars, forks, contributors_count, created_year, and primary_language .
- **Regression Target Derivation:** For the regression task, a specialized target was created: the byte-size of the most dominant secondary language (excluding Python) for all Python repositories (secondary_bytes). This target was Log-transformed (np.log1p) to mitigate the skewness of the distribution, which significantly improved prediction accuracy.
- **Encoding and Splitting:** Categorical features were encoded using LabelEncoder. For the regression model, the final feature matrix X and target vector y (log-transformed secondary_bytes) were split into 80% for training and 20% for testing .

ii. Clustering Approach

Unsupervised learning was used to identify natural groupings among all repositories.

Methodology: K-Means was employed to identify spherical, dense clusters, while DBSCAN was used to explore potential non-spherical, density-based groupings. All four input features were standardized prior to clustering.

K-Means Results: The Elbow Method suggested an optimal $K=4$. The resulting Silhouette Score was moderate (0.383), indicating a degree of overlap, but the visualization clearly separated outlier clusters composed of exceptionally high-star/high-fork repositories from the main, low-engagement cluster.

DBSCAN Results: DBSCAN primarily identified a single core cluster and was unable to meaningfully partition the data into more than one group, classifying only 1.10% of data points as noise. This confirms that the majority of the AI repositories form one dense, interconnected group, with the most successful projects existing as distinct, sparse outliers in the feature space.

iii. Regression Results

The goal of the regression task was to predict the byte size of the secondary language in Python-dominant repositories using success metrics and repository characteristics.

Model	Mean Squared Error (MSE)	R2 Score
Baseline (Mean Prediction)	23.82	-0.000
Linear Regression	15.78	0.337
Decision Tree Regressor	3.70	0.845
Random Forest Regressor	2.11	0.912

The Random Forest Regressor was the best-performing model, demonstrating a strong capacity to predict the complexity of the secondary language used in a Python project based on its success metrics. The high R^2 score of 0.912 indicates that the model's input features explain 91.2% of the variance in the log-transformed secondary language size. This is a significant improvement over the linear and baseline models.

The success of the Random Forest model suggests a non-linear but predictive relationship: highly successful Python AI repositories tend to have a predictable secondary codebase composition (e.g., dominated by Shell or Jupyter Notebook), which is quantifiable based on their stars, forks, and contributors_count.

iv. Hypothesis Testing and Findings

To address the project's central objective, one-sample and two-sample t-tests were performed at an $\alpha = 0.05$ significance level to evaluate the shift in Python usage and popularity following ChatGPT's release (November 30, 2022).

Key Findings:

Shift in Python Proportion: Python's proportion among all new AI repositories significantly declined from 47.86% to 40.41% (a 7.45 percentage point drop). This is interpreted as a broadening of the AI ecosystem, where new developers (likely from web-development

backgrounds) began creating AI-related projects in languages like JavaScript and TypeScript, enabled by LLMs.

Surge in Python Popularity: The average stars received by Python repositories significantly increased by 168% after the release (Mean: 2,171 $\rightarrow$ 5,812; $t = -7.46$, $p < 0.001$).

Conclusion: Quality Over Quantity: The decrease in Python's overall proportion combined with the huge increase in average popularity indicates a transformation of Python's role from a ubiquitous foundation to a highly selective platform for high-impact LLM projects.

Statistical Confidence:

A Power Analysis confirmed the reliability of these findings. The study's sample sizes (1,218 and 1,036) exceeded the minimum required N (154 per group) for 80% power by 6 to 7-fold. This ensured the highly significant increase in stars was detected with high confidence ($p < 0.001$).

4. Conclusion and Recommendations

The analysis of AI-related GitHub repositories spanning 2020-2025 provides strong evidence that the release of ChatGPT fundamentally altered the open-source AI development landscape.

- **Python's Evolved Role:** While the overall usage share of Python in AI declined, the mean popularity (stars) of the remaining Python repositories surged by a statistically significant 168%. This suggests that Python is no longer just the most common language but has become the dominant, high-impact platform for next-generation LLM projects.
- **Ecosystem Expansion:** The proportional decline of Python indicates that LLMs have lowered the barrier to entry for non-Python developers, broadening the AI ecosystem with increased use of languages like JavaScript and TypeScript.
- **Modelling Success:** The Random Forest Regressor successfully predicted the complexity of the secondary language used in Python projects ($R^2=0.912$), confirming that project success metrics (stars, forks) are strongly correlated with the composition of the codebase.

Recommendations for Future Work

Balance Data: The classification models in previous phases struggled (accuracy near baseline) due to data imbalance. Future work should employ oversampling (SMOTE) or use weighted classification metrics (like F1-score or AUC) rather than simple accuracy to better handle less common languages.

Additional Features: Incorporating Natural Language Processing (NLP) on the description and topics features could capture the intent of the repository (e.g., framework vs. application) which may be a stronger predictor of language choice and success .